

Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

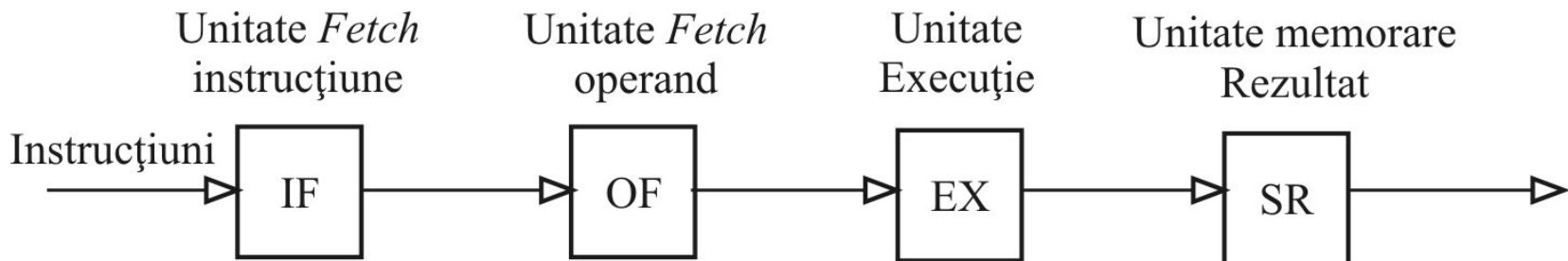
Descriu situații normale: datele utilizate de anumite instrucțiuni depind de datele create de alte instrucțiuni. Altfel spus, datele de ieșire (rezultatele) unor instrucțiuni devin date de intrare (operandii) altor instrucțiuni.

A. Dependența reală (hazardul "*read-after-write*")

Fie secvența:

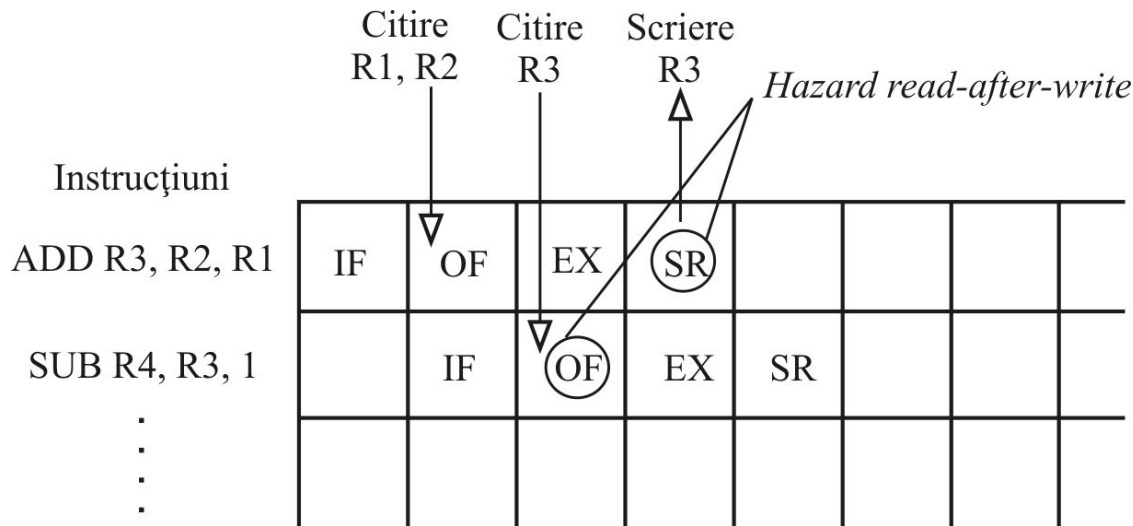
1. ADD R3,R2,R1 ; $R3 = R2 + R1$
2. SUB R4,R3,1 ; $R4 = R3 - 1$

Ar fi incorect ca instrucțiunea 2 să citească R3 înainte ca instrucțiunea 1 să scrie rezultatul în R3. Între instrucțiunea 1 și instrucțiunea 2, există deci o dependență reală de date numită și hazard "*read-after-write*" (RAW). Hazardul RAW apare atunci când rezultatul obținut într-o instrucțiune devine operandul uneia dintre instrucțiunile următoare. Vom considera procesarea celor 2 instrucțiuni într-un *pipeline* pe 4 nivele:

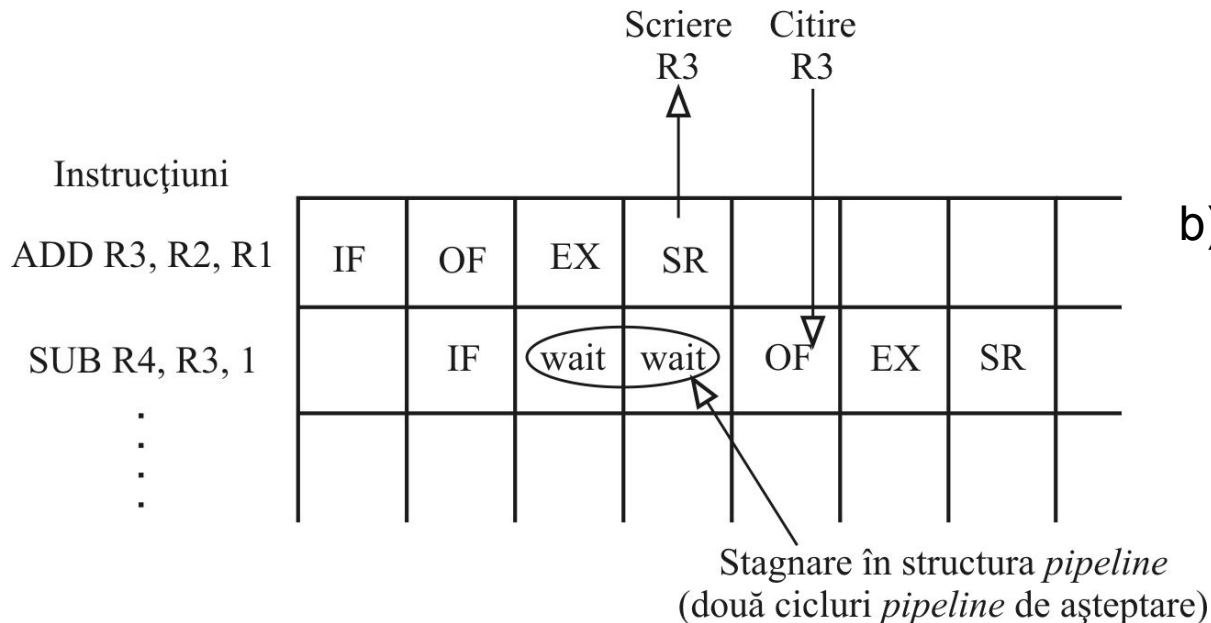


Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date / Hazardul RAW



a) Procesarea celor două instrucțiuni fără a ține cont de hazardul RAW (instrucțiunea SUB va prelua incorect operandul din R3 !)



b) Procesarea celor două instrucțiuni ținând cont de hazardul RAW (instrucțiunea SUB va prelua corect operandul din R3 dar apare o penalitate de 2 cicluri *pipeline*)

Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

A. Dependența reală (hazardul RAW)

- Dacă între cele două instrucțiuni aflate în hazard RAW se intercalează alte instrucțiuni (independente) astfel încât *fetch*-ul operandului pentru instrucțiunea SUB să se facă după scrierea rezultatului instrucțiunii ADD, atunci hazardul RAW va fi soluționat fără diminuarea performanțelor procesorului. Compilatoarele cu reorganizare au această sarcină pe care o soluționează prin reordonarea instrucțiunilor din cadrul programului, fără a afecta logica acestuia.
- Există situații în care nu pot fi găsite suficiente instrucțiuni independente și de aceea unitatea de control trebuie să fie dotată cu capacitatea de a detecta și rezolva aceste hazarduri la nivel *hardware* (prin inserare de cicluri *wait*).
- Hazardurile RAW se manifestă în toate procesoarele *pipeline*-izate, inclusiv în cele care lansează instrucțiunile în execuție în ordinea din program (*in-order issue*).

Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

B. Antidependența (hazardul *write-after-read*)

Fie secvența:

1. ADD R3,R2,R1 ; $R3 = R2 + R1$
2. SUB R2,R3,1 ; $R2 = R3 - 1$

În afară de hazardul RAW pe registrul R3, între cele două instrucțiuni mai apare și un hazard *write-after-read* (WAR) pe R2.

În cele mai multe structuri *pipeline*, unitatea de *fetch* operand este plasată în cascada *pipeline* înaintea unității de scriere.

De aceea, hazardurile WAR:

- nu creează probleme în implementările *in-order issue*.
- devin probleme în implementările *out-of-order issue*.

Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

C. Dependența de ieșire (hazardul *write-after-write*)

Fie secvența:

1. ADD R3,R2,R1 ; $R3 = R2 + R1$
2. SUB R2,R3,1 ; $R2 = R3 - 1$
3. ADD R3,R2,R5 ; $R3 = R2 + R5$

În afară de hazardurile RAW pe R3 (între primele două instrucțiuni) și respectiv pe R2 (între ultimele două) și WAR pe R2 (între primele două instrucțiuni), apare și un hazard *write-after-write* (WAW) pe R3, între instrucțiunile 1 și 3. Instrucțiunea 1 trebuie să genereze rezultatul aferent în R3, înainte ca instrucțiunea 3 să genereze propriul rezultat în R3 (cele două scrieri în R3 nu trebuie inversate!).

Hazardul WAW:

- nu creează probleme în implementările *in-order-issue* (dacă operațiile de scriere sunt controlate de o singură unitate din structura *pipeline*).
- devin probleme în implementările *out-of-order issue*.

Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

D. Detecția hazardurilor

Între 2 instrucțiuni, I_i urmată de I_j , există:

- hazard RAW dacă: $O(I_i) \cap I(I_j) \neq \emptyset$
- hazard WAR dacă: $I(I_i) \cap O(I_j) \neq \emptyset$
- hazard WAW dacă: $O(I_i) \cap O(I_j) \neq \emptyset$

$O(I_i)$ - indică setul de *output*-uri ale instrucțiunii I_i (registru/registrele destinație aferent/aferente instrucțiunii I_i)

$I(I_i)$ - indică setul de *input*-uri ale instrucțiunii I_i (registrele sursă aferente instrucțiunii I_i)

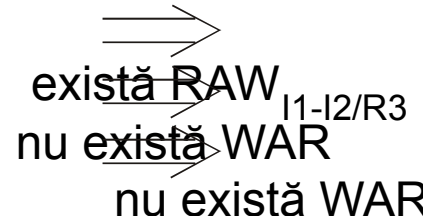
Pentru exemplificare, presupunem următoarea secvență:

I1: ADD R3,R2,R1 ; R3 = R2 + R1

I2: SUB R5,R3,1 ; R5 = R3 - 1

care va fi procesată într-o structură *pipeline*.

Pe baza celor 3 condiții obținem:

$O(I1) = (R3)$	$\xrightarrow{\quad}$	$O(I1) \cap I(I2) = (R3) \cap (R3) = R3$	
$O(I2) = (R5)$	$\xrightarrow{\quad}$	$I(I1) \cap O(I2) = (R1,R2) \cap (R5) = \emptyset$	
$I(I1) = (R1,R2)$	$\xrightarrow{\quad}$	$O(I1) \cap O(I2) = (R3) \cap (R5) = \emptyset$	
$I(I2) = (R3)$			

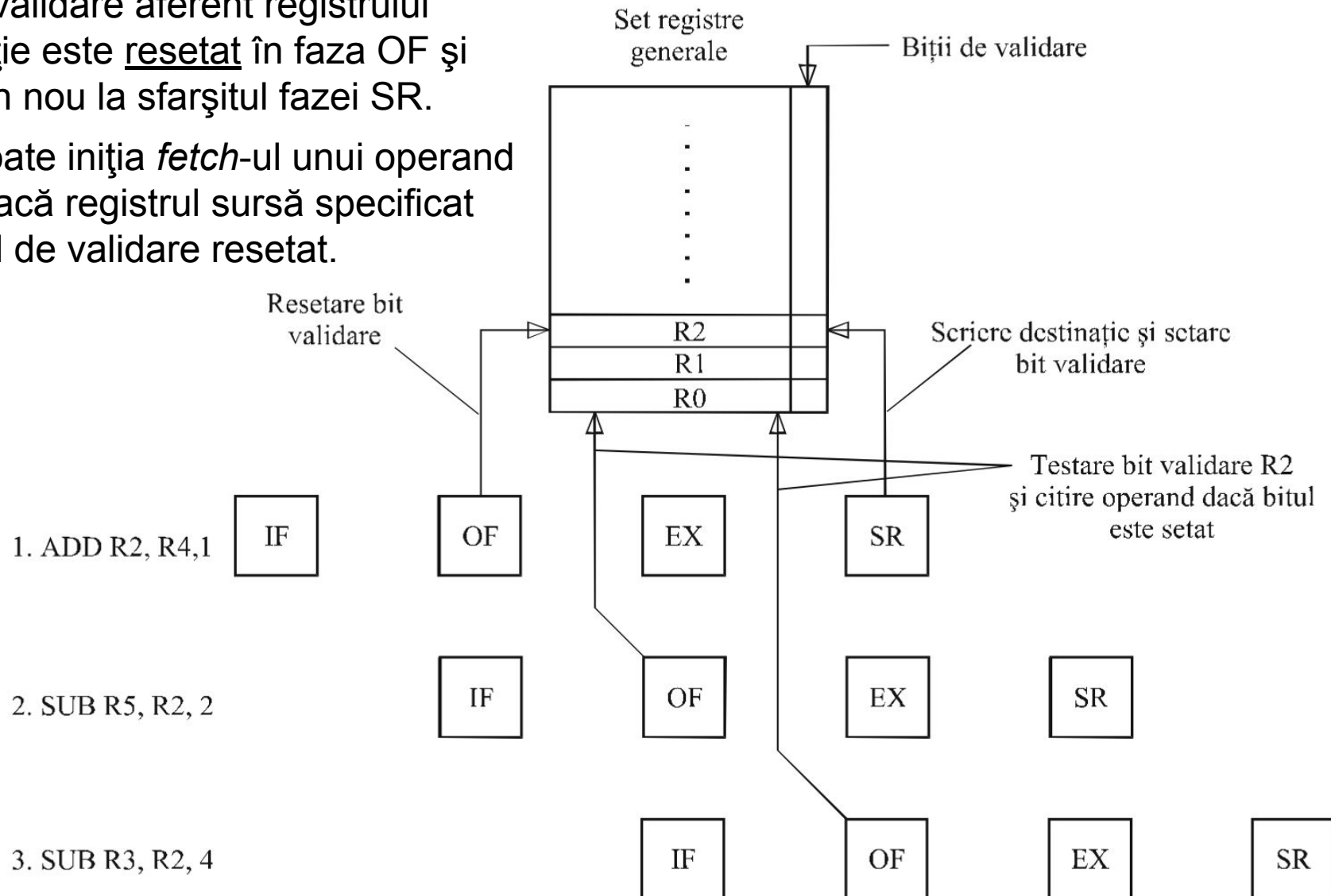
În consecință, între cele 2 instrucțiuni **I1** și **I2** există un hazard RAW pe R3

Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

E. Detecția și controlul hazardurilor cu ajutorul biților de validare

- bitul de validare aferent registrului destinație este resetat în faza OF și setat din nou la sfârșitul fazei SR.
- nu se poate iniția *fetch*-ul unui operand sursă dacă registrul sursă specificat are bitul de validare resetat.



Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

F. Metoda tablei de marcaj ("*scoreboard*")

Metoda tablei de marcaj ("*scoreboard*") utilizează 3 tabele implementate în *hardware*:

- tabela de stare a instrucțiunilor
- tabela de stare a unităților funcționale
- tabela de stare a registrelor destinație

Fie un procesor cu 4 unități funcționale distincte dedicate execuției instrucțiunilor:

- unitatea *load/store* pentru execuția instrucțiunilor *load* și respectiv *store*
- unitatea de înmulțire pentru execuția instrucțiunilor MUL
- unitatea de adunare 1 pentru execuția instrucțiunilor ADD și SUB
- unitatea de adunare 2 pentru execuția instrucțiunilor ADD și SUB

și care execută următoarea secvență de program:

```
LD R1,A          ; R1 = A
LD R2,B          ; R2 = B
LD R3,C          ; R3 = C
LD R4,D          ; R4 = D
MUL R5,R1,R2     ; R5 = R1 * R2
ADD R2,R3,R4     ; R2 = R3 + R4
ADD R2,R2,R5     ; R2 = R2 + R5
```


Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

F. Metoda tablei de marcaj ("*scoreboard*")

Instrucțiune	<i>Fetch</i> Instrucțiune încheiat	<i>Fetch</i> operand încheiat	Execuție încheiată	Scriere rezultat încheiată
LD R1,A	da	da	da	da
LD R2,B	da	da	da	da
LD R3,C	da	da	da	da
LD R4,D	da	da		
MUL R5,R1,R2	da	da		
ADD R2,R3,R4	da			
ADD R2,R2,R5				

a) Tabela de stare a instrucțiunilor

Număr unitate	Denumire unitate	Busy	Registrul destinație Rd	Registrele sursă			
				Rs1	Ready	Rs2	Ready
1	Load / Store	da	R4				
2	Înmulțitor	da	R5	R1	da	R2	da
3	Sumator 1	da	R2	R3	da	R4	nu
4	Sumator 2	nu					

b) Tabela de stare a unităților funcționale

Registre	R0	R1	R2	R3	R4	R5		Rn
Număr unitate			3		1	2		

c) Tabela de stare a registrelor destinație

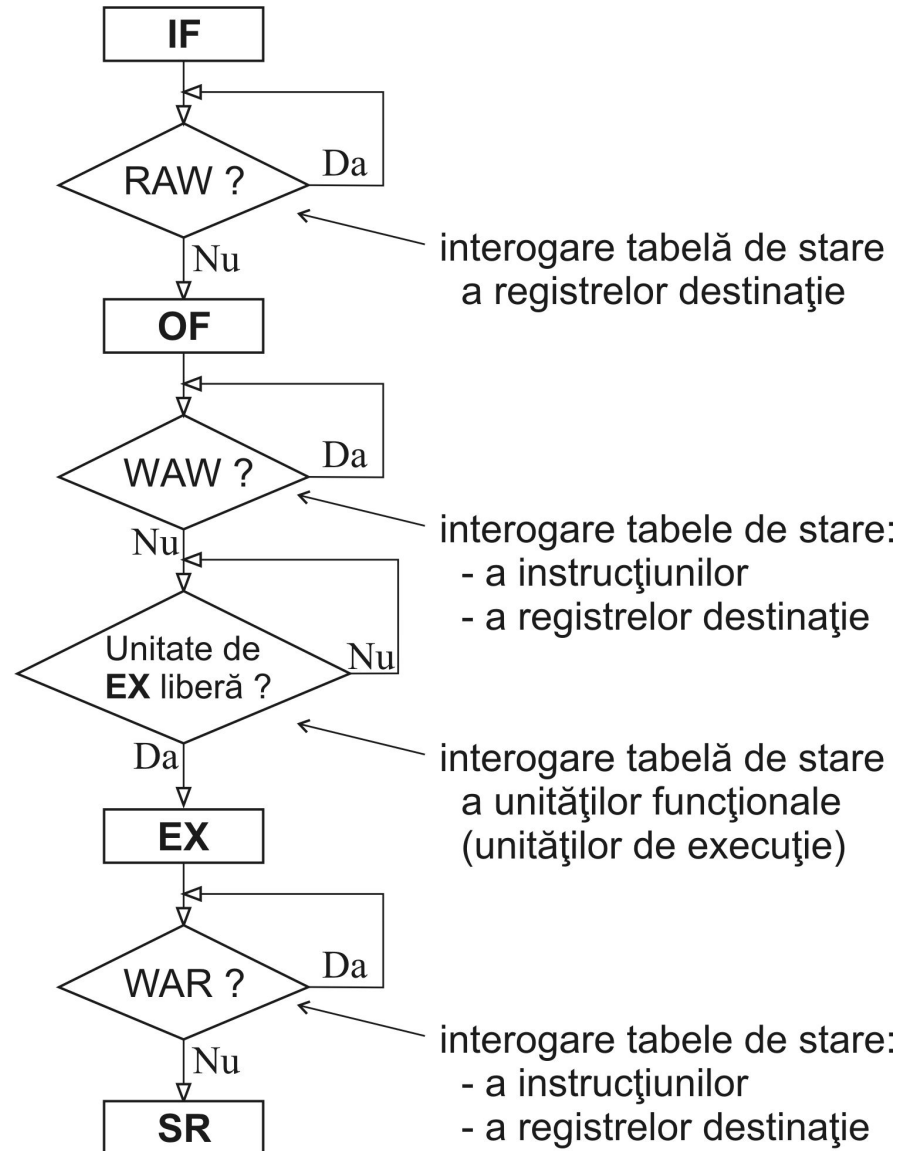
Fig. 5.25. Starea tablei de marcaj după *fetch*-ul primei instrucțiuni ADD

Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

F. Metoda tabelii de marcaj ("scoreboard")

Algoritmul de monitorizare a
tabelilor de marcaj pentru
dectecția și controlul hazardurilor:



Hazardurile în *pipeline*-urile de instrucțiuni

3. Hazardurile de date (dependențele de date)

F. Metoda tablei de marcaj ("*scoreboard*")

Instrucțiune	<i>Fetch</i> Instrucțiune încheiat	<i>Fetch</i> operand încheiat	Execuție încheiată	Scriere rezultat încheiată
LD R1,A	da	da	da	da
LD R2,B	da	da	da	da
LD R3,C	da	da	da	da
LD R4,D	da	da	da	da
MUL R5,R1,R2	da	da	da	
ADD R2,R3,R4	da	da	da	da
ADD R2,R2,R5	da			

a) Tabela de stare a instrucțiunilor

Număr unitate	Denumire unitate	Busy	Registrul destinație Rd	Registrele sursă			
				Rs1	Ready	Rs2	Ready
1	Load / Store	nu					
2	Înmulțitor	da	R5	R1	da	R2	da
3	Sumator 1	nu					
4	Sumator 2	da	R2	R2	da	R5	nu

b) Tabela de stare a unităților funcționale

Registre	R0	R1	R2	R3	R4	R5		Rn
Număr unitate			4			2		

c) Tabela de stare a registrelor destinație

Fig. 5.26. Starea tablei de marcaj după *fetch*-ul celei de-a 2-a instrucțiuni ADD.

Hazardurile în *pipeline*-urile de instrucțiuni

4. Accelerarea prin transfer direct (*Forwarding*)

A. *Forwarding* la nivelul compilatorului

Trei tipuri de *forwarding* pot fi identificate la nivel *software*:

- ***store-fetch forwarding*** (*fetch*-ul unui operand, care a fost memorat într-o locație de memorie și care încă se mai află și într-un registru, este eliminat preluând operandul direct din registru):

ST [R1],R2 $\longrightarrow$ ST [R1],R2
LD R3,[R1] $\longrightarrow$ ADD R3,R2,R0 ; $R3 \leftarrow R2 + 0$ (*move* R2 în R3)

- ***fetch-fetch forwarding*** (*n* accese la aceeași locație de memorie sunt convertite într-un acces la locația respectivă, prin care locația este transferată într-un registru, urmat de *n-1* accese la registru):

LD R2,[R1] $\longrightarrow$ LD R2,[R1]
LD R3,[R1] $\longrightarrow$ ADD R3,R2,R0 ; $R3 \leftarrow R2 + 0$

- ***store-store overwriting*** (se reduc scrierile succesive în aceeași locație de memorie):

ST [R1],R2 $\longrightarrow$ ST [R1],R3
ST [R1],R3 $\longrightarrow$ —

Hazardurile în *pipeline*-urile de instrucțiuni

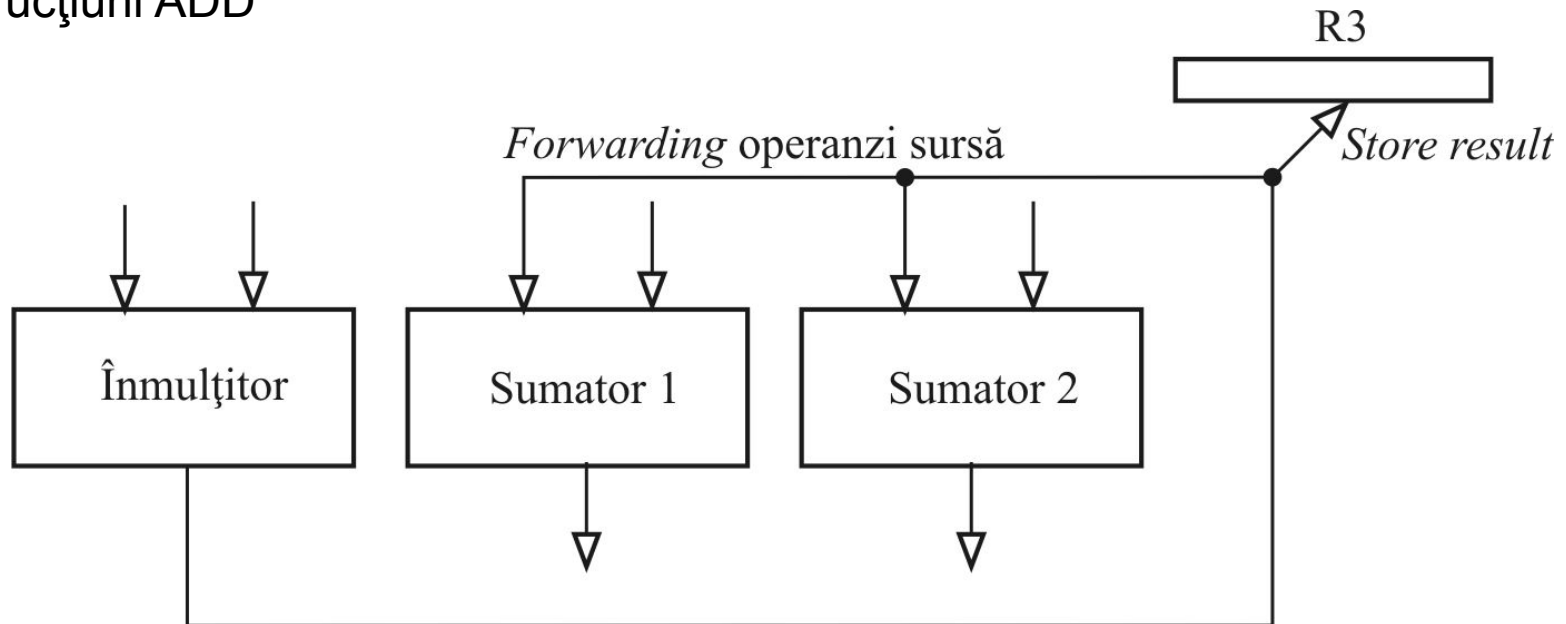
4. Accelerarea prin transfer direct (*Forwarding*)

B. *Forwarding* la nivel *hardware* (*forwarding* intern)

Forwarding-ul poate fi extins în sensul transmiterii directe a unui rezultat către mai multe instrucțiuni. De exemplu pentru următoarea secvență de instrucțiuni:

```
MUL R3, R2, R1      ; R3 = R2 * R1
ADD R4, R3, 8        ; R4 = R3 + 8
ADD R5, R3, 12       ; R5 = R3 + 12
```

unitatea de înmulțire va genera rezultatul pentru R3, rezultat care va fi transmis direct (*forwarding*) celor două sumatoare responsabile cu execuția celor două instrucțiuni ADD



Forwarding utilizând unități funcționale multiple